Procedure Selection versus Workflow Structuring in Local RAG for Institutional SOP Question Answering: An Ablation Study

Cipta Fikri Wiratama[a], Indra Waspada[a,*], Dhena Kamalia Fu'adi[a], Ramona Ramli[b,*], Henri Tantyoko[a], Sandy Kurniawan[a], Satriawan Rasyid Purnama[a]

[a] *Department of Informatics, Faculty of Science and Mathematics, Universitas Diponegoro, Semarang, 50275, Indonesia*
[b] *School of Computer Sciences, Universiti Sains Malaysia, Pulau Pinang, 11800 USM, Malaysia*

Abstract

Standard operating procedures (SOPs) in university administration require answers that preserve step order, responsible actors, and required documents. Locally hosted language models avoid commercial API fees and protect data privacy, but small models are easily distracted by passages retrieved from lexically overlapping procedures. We investigate a workflow-aware Retrieval-Augmented Generation (RAG) pipeline that pairs rule-based procedure selection with directed workflow graphs encoding chronological steps and responsible actors. We compare it with nine alternative configurations on a verified 40-question benchmark over seven Indonesian institutional SOPs, using a local generator (`gemma4:e2b`, 5.1B parameters). The complete Workflow GraphRAG pipeline achieves 0.955 key-fact recall (KFR) and 1.000 step coverage, improving on naive RAG (0.712 KFR; Holm-adjusted $p = 0.053$) and a custom entity-graph baseline (0.698 KFR; Holm-adjusted $p = 0.034$). However, a selection-matched text ablation that supplies the same selected SOP as raw text achieves 0.962 KFR, and a paired Wilcoxon test detects no significant recall advantage from graph serialization ($p = 0.59$). Non-fine-tuned base embeddings raise naive RAG to 0.910 KFR, revealing strong baseline sensitivity. Workflow GraphRAG shows zero topological inversions (TIR = 0.000), whereas entity-graph retrieval (TIR = 0.083) and full-context prompting (TIR = 0.027) introduce ordering errors, and it uses 44% fewer prompt tokens than full-context prompting with comparable recall. These results show that procedure-level selection is the primary driver of factual recall in concise procedural corpora, while workflow graphs provide chronological formatting integrity and token efficiency.

Keywords: Retrieval-augmented generation, GraphRAG, Standard operating

*Corresponding author
Email addresses: `fikriw@students.undip.ac.id` (Cipta Fikri Wiratama), `indrawaspada@lecturer.undip.ac.id` (Indra Waspada), `ramona@usm.my` (Ramona Ramli)

procedures, Workflow representation, Small language models, Ablation study, Evaluation metrics

---

Word count: 7,270 words, excluding references.

1. Introduction

Universities run routine student services, such as academic leave, course registration, tuition-payment postponement, and credential legalisation, through Standard Operating Procedures (SOPs). Although these procedures are documented, administrative staff spend substantial time answering repeated questions about required steps, responsible units, and prerequisites, which makes question answering with Large Language Models (LLMs) attractive.

Cloud-hosted proprietary models, however, add per-query costs and expose institutional records and student data to third-party infrastructure. Small open-weights models running on local hardware avoid both problems, but models of 2B to 5B parameters reason less reliably and are more easily distracted by irrelevant context than large cloud services [10]. In administrative answers this matters: transposing two steps, naming the wrong official, or omitting a mandatory form sends a student to the wrong desk.

Retrieval-Augmented Generation (RAG) grounds generation in retrieved passages [1, 2]. For short institutional SOPs, standard chunking yields whole-document chunks, so top-k retrieval returns several distinct SOPs that share vocabulary (e.g., Dean, Academic Office, verification, approval signatures). The small generator must then separate competing procedures, which risks procedural cross-contamination and wrong step orderings. Graph-based RAG [4, 11] mostly builds graphs of named entities and relational triples [3, 12, 13], which do not represent execution order or step-level responsibility.

We investigate *Process-Aware Workflow GraphRAG*, which models each SOP as a directed workflow graph [16] whose step nodes record the performing actor, action, required documents, and deliverables. A transparent rule-based selector identifies the target SOP, and its ordered steps, together with supporting text chunks, are passed to a local generator (`gemma4:e2b`). Gains from graph-enhanced retrieval are often attributed to graph modelling without testing whether they come from *retrieving the correct document* or from *presenting it as a graph*. If selection is the main bottleneck, the raw text of the selected SOP may achieve comparable accuracy without graph curation. We therefore ask four research questions:

- **RQ1 (Answer quality):** How does the workflow-aware pipeline compare with chunk retrieval and an entity-graph baseline in factual completeness, step coverage, and order and actor correctness?
- **RQ2 (Selection vs. structuring):** What does graph-derived workflow structuring add beyond selecting the same SOP and supplying its original text?

- **RQ3 (Refusal):** How do the systems behave on out-of-scope or unanswerable questions, and which mechanisms govern their refusals?
- **RQ4 (Efficiency):** What are the prompt-token and latency trade-offs relative to retrieval baselines and full-context prompting?

This study makes four contributions:

1. **Workflow-aware procedural RAG pipeline:** rule-based SOP identification with directed workflow-graph serialization for a small local generator.
2. **Selection-versus-structure ablation:** *Selection-Matched Text RAG* (logged as *SOP-Oracle RAG*), which keeps selection and supporting retrieval fixed but replaces the workflow block with raw SOP text.
3. **Deterministic procedural metrics:** LLM-free Topological Inversion Rate (TIR) and Actor Mismatch Rate (AMR), used with step coverage and key-fact recall (KFR) and checked by a manual audit of 60 sampled rows.
4. **Controlled comparison:** 10 systems on a verified 40-question benchmark, showing that the ablation matches Workflow GraphRAG (0.962 vs. 0.955 KFR; paired Wilcoxon $p = 0.59$) and that embedding configuration strongly affects standard RAG.

2. Related work

2.1. Retrieval-augmented generation

RAG conditions generation on retrieved documents [1]. Naive RAG indexes chunks with dense embeddings and injects the top-k neighbours into the prompt [2]; hybrid retrieval adds sparse BM25 scoring [5] merged by reciprocal rank fusion [6]; agentic methods such as ReAct [7] interleave reasoning with tool calls; and Corrective RAG (CRAG) [8] and Self-RAG [9] grade retrieved evidence and rewrite queries or fall back when it is insufficient. All of these pass unstructured prose to the generator, leaving step order and actor attribution to be inferred. Supplying a whole collection in a long prompt avoids retrieval but raises token cost and risks weaker attention to mid-context content [10].

2.2. Graph-based RAG architectures

Graph-based RAG adds explicit relations between pieces of evidence [4, 11]. Microsoft GraphRAG [3] extracts entity-relation networks with LLM prompting and summarises hierarchical communities for global queries; LightRAG [12] uses dual-level entity and relation indexing to reduce cost; and HippoRAG [13] applies Personalized PageRank over entity graphs for multi-hop retrieval. These graphs encode relational triples among named entities and have no native notion of chronological precedence, branching, or execution state.

2.3. Process modelling and procedural representation

Business Process Management represents procedures as directed workflows [16]. BPMN models activities, sequence flows, swimlanes for actor roles, and gateways for branching [17], and process models can be extracted from natural-language descriptions [18]. Whereas process models usually serve simulation, compliance checking, or orchestration, we use workflow graphs as a retrieval and prompt-structuring substrate for generation.

2.4. Emerging order-aware and procedural RAG

Two concurrent studies model procedural order explicitly. SOPRAG [14] retrieves industrial SOPs with flow-graph experts for step sequences and entity-graph experts for tools and components, and Order-Aware Hypergraph RAG [15] models transitions over hyperedges to capture order across multi-step interactions. Both target large-scale settings with learned graph indexing, and neither isolates the contribution of selecting the right procedure from that of structuring it. Our study addresses this gap for small local models over institutional workflows. Table 1 contrasts the context units that each method family passes to the generator.

Table 1: Structural characteristics of context units provided to the generator across retrieval paradigms. "Implicit" denotes that information is present in prose but requires model inference.

Method family	Context unit passed to generator	Step sequence	Actor per step
Naive / Hybrid / Agentic / CRAG [1, 6, 7, 8]	Unstructured text chunks	Implicit	Implicit
Entity-graph GraphRAG [3, 12]	Entity-relation triples & text	Not represented	Relational triples
Workflow GraphRAG (this work)	Directed workflow block & chunks	Explicit	Explicit
Selection-Matched Text (ablation)	Whole selected SOP text & chunks	Implicit	Implicit

3. Problem setting and procedural error classes

3.1. Institutional SOP corpus

The corpus is the complete set of official student-service SOPs published by the Faculty of Science and Mathematics (FSM), Universitas Diponegoro (Table 2): 7 one-page Indonesian documents on academic standing, tuition processing, and student activities, with 44 numbered steps and about 1,097 words. Every procedure is a strictly linear sequence of steps with no decision gateways or concurrent branches. All documents share one layout (Figure 1): a title, a purpose sentence that may state the maximum processing time, and numbered steps whose grammatical subject is the performing role, optionally followed by

Table 2: Characteristics of the 7 FSM Universitas Diponegoro SOP documents. "PDF steps" denotes numbered steps in official source documents; "Graph steps" denotes nodes in the curated workflow graph.

SOP Document	PDF steps	Graph steps	Characters	Words
Academic leave (Cuti Akademik)	8	8	1,573	205
Reactivation after leave (Aktif Kembali)	3	3	597	84
Diploma/transcript legalisation (Legalisir)	6	6	1,640	208
Course registration (Pengisian IRS)	6	6	1,189	163
Late tuition payment (Keterlambatan UKT)	11	11	1,415	173
Scholarship recommendation (Beasiswa)	5	5	1,130	142
Student activity proposal (Proposal Ormawa)	5	5	997	122
Total	44	44	8,541	1,097

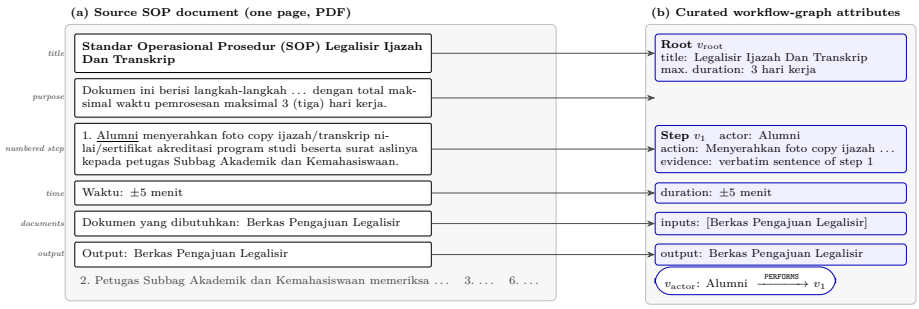


Figure 1: Anatomy of an FSM SOP document and its mapping to workflow-graph attributes, shown for step 1 of the legalisation SOP. The underlined grammatical subject becomes the performing actor, the optional *Waktu*, *Dokumen yang dibutuhkan*, and *Output* lines become duration, input, and output attributes, and the step sentence is stored verbatim as evidence.

Waktu (time), *Dokumen yang dibutuhkan* (required documents), and *Output* lines.

The steps are performed by 15 roles spanning the student, study-programme, faculty, and university levels (Figure 2). Students perform 21 of the 44 steps and act in six of the seven SOPs, and several faculty roles, such as the Vice Dean for Academic and Student Affairs and the Resources Sub-division, act in more than one procedure. Different SOPs therefore share actors as well as vocabulary.

3.2. Procedural correctness requirements and error taxonomy

A correct administrative answer lists the mandatory steps in execution order, attributes each action to the right official, and states prerequisites and deliverables. We define three failure modes:

- **E1 (Step Order Inversion):** downstream actions are presented before their prerequisites (e.g., Dean approval before Academic Sub-division verification).
- **E2 (Actor Mismatch):** a task is assigned to the wrong official or office (e.g., stating that the Dean checks attachments that Academic Sub-division staff verify).

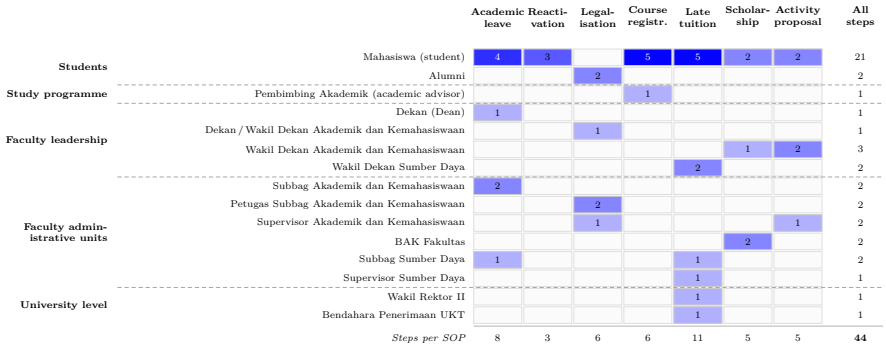


Figure 2: Organisational roles that perform SOP steps, grouped by administrative level, and the number of steps each performs per SOP (`PERFORMS` edges; darker cells denote more steps). Roles that only approve, sign, or receive copies within a step, such as the Head of Study Programme in the leave procedure, are recorded in action descriptions and not counted.

- **E3 (Procedural Cross-Contamination):** steps or prerequisites from an unrelated SOP enter the answer (e.g., tuition-settlement steps in an academic-leave answer).

Table 3 maps each error class to the metric that measures it and to the design component intended to address it.

Table 3: Procedural error classes, the metric used to measure each, and the component of Workflow GraphRAG designed to address it.

Error class	Measured by	Design component	Reported in
E1 Step order inversion	TIR (Eq. 2); step coverage	`NEXT_STEP` traversal and ordered serialization	Table 7
E2 Actor mismatch	AMR (Eq. 3)	Per-step actor attribute via `PERFORMS`	Table 7; Section 5.6
E3 Procedural cross-contamination	KFR, cross-SOP and single-SOP questions	Rule-based SOP selection (Eq. (1))	Tables 8, 9

3.3. Corpus chunking and procedural cross-contamination

Documents are chunked with a 1,700-character window and a 100-character overlap. Because every SOP in Table 2 is under 1,650 characters, each maps onto exactly one chunk ($N = 7$), so top-k retrieval with $k = 3$ always places three complete SOPs in the prompt. Given the shared vocabulary and actors described above, the challenge in this setting is not reconstructing fragmented passages but preventing cross-contamination (E3) and preserving order and actor fidelity (E1, E2).

4. Proposed method and ablation design

4.1. System overview

Process-Aware Workflow GraphRAG combines offline curation with an online pipeline (Figure 3). Offline, the SOPs are curated into directed workflow graphs and indexed in a dense vector store. Online, a rule-based scorer selects the target SOP, its ordered steps are serialized into a workflow block, supporting chunks are retrieved, and a local generator produces the answer.

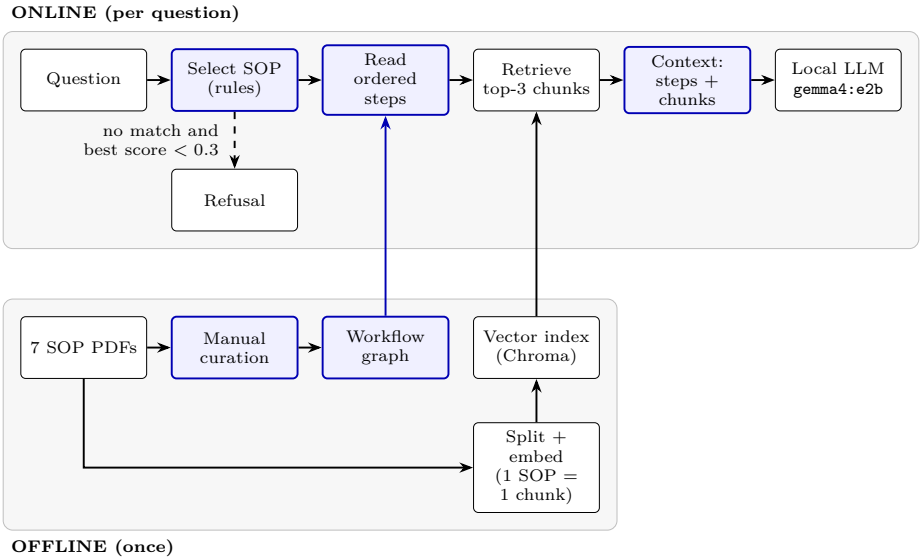


Figure 3: Architecture of Process-Aware Workflow GraphRAG. Highlighted nodes indicate components introduced: rule-based SOP selection and ordered workflow graph extraction. The underlying vector index, embedding model, prompt template, and generator are shared across evaluated systems.

4.2. Workflow graph construction

Each procedure p is a directed graph $G_p = (V_p, E_p)$ implemented in NetworkX, with three node classes:

1. **Root node (v_{root}):** the procedure, with its official title, aliases (e.g., "cuti kuliah" for academic leave), and maximum duration where stated.

2. **Step nodes (v_i), $i \in \{1, \ldots, n_p\}$:** the performing actor, action, required input documents, outputs, duration, and a verbatim quote from the source PDF.

3. **Actor nodes (v_{actor}):** organisational roles shared across procedures.

Edges are of three types: `HAS_STEP`(v_{root}, v_i) links the root to each step, `NEXT_STEP`(v_i, v_{i+1}) encodes execution precedence, and `PERFORMS`(v_{actor}, v_i)

attributes each step to its actor. The graph was built by hand from the 7 PDFs and contains 7 root, 44 step, and 15 actor nodes connected by 125 directed edges. The primary actor of a step is the grammatical subject of its instruction; consulted or secondary signatories are recorded in the action description. Verification scripts confirm that every quote matches the source verbatim and that step counts equal those of the PDFs. Figure 4 contrasts this representation with the entity-graph baseline.

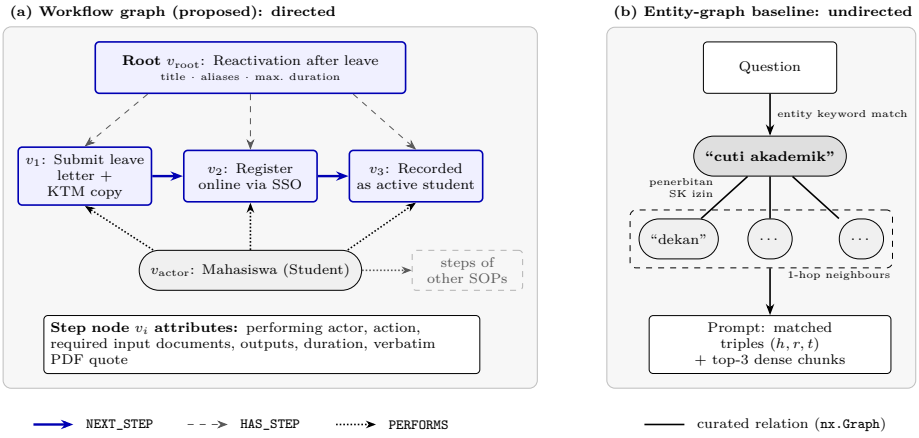


Figure 4: Graph representations compared in this study. (a) The proposed directed workflow graph for the *Reactivation after leave* SOP; the full graph has 7 root, 44 step, and 15 actor nodes and 125 edges. (b) The undirected entity-graph baseline: question entities are matched against 17 curated triples, and 1-hop triples are prepended to the top-3 dense chunks.

4.3. Rule-based SOP selection

Given question q, the target procedure p^* is identified with a transparent scoring function:

$$s(q,p) = 15\mathbb{1}[title_p \subset q] + \sum_{a \in aliases_p} g(q,a) + \sum_{v \in steps_p} h(q,v), \tag{1}$$

where $g(q,a) = 8$ if alias a occurs as an exact substring of q, or 2 per constituent word ($|w| > 2$); and $h(q,v) = 1$ if q contains an explicit procedural cue (Indonesian keywords for "how", "procedure", "requirements", "flow", or "documents") and shares a content word ($|w| > 4$) with the action of step v. The highest-scoring procedure is selected if $s(q,p^*) \geq 3$. A priority rule maps questions containing keywords for both "active" and "leave" to *Reactivation after leave*, resolving confusion between those two procedures. Matching is case-insensitive; the rules are deterministic and inspectable but tailored to this 7-SOP collection.

4.4. Context construction and supporting retrieval

For the selected p^*, step nodes are read by a topological traversal along `NEXT_STEP` edges and serialized into an ordered block that lists step index, actor, action, required documents, outputs, and duration (Figure 5).

OFFICIAL WORKFLOW: REACTIVATION AFTER LEAVE (AKTIF KEMBALI)
Maximum processing duration: Not specified in document

Step 1. Actor: Mahasiswa (Student)
Action: Submit academic leave permit letter from prior semester and student card copy to Head of Study Programme and Academic Sub-division.
Required Documents: Academic leave permit letter, Student card (KTM) copy
Duration: ±5 minutes

Step 2. Actor: Mahasiswa (Student)
Action: Register online for reactivation through university Single Sign-On (SSO) portal.
...

Step 3. Actor: Mahasiswa (Student)
Action: Registered as active course participant in academic database.
...

Supporting Documents: *[Top-3 dense chunks retrieved from vector store]*

Figure 5: Illustrative context block produced for reactivation inquiry (translated from Indonesian). Each step explicitly details performing actor, action, and required documentation.

For supplementary details such as notes and URLs, the query is expanded with the SOP title and the actions of its first three steps, and $k = 3$ chunks retrieved with it from the shared Chroma index are appended after the workflow block. Because each chunk is a whole SOP, these supporting chunks may contain other procedures, which we analyse in Section 7.

4.5. Generation and refusal mechanism

The context and question are inserted into a standard prompt template that instructs the model to answer in Indonesian, use only the provided facts, and abstain when evidence is insufficient; inference runs locally with `gemma4:e2b` via Ollama. Refusal is triggered deterministically before generation if (1) no SOP is selected ($s(q, p^*) < 3$) and (2) the maximum cosine similarity between q and the retrieved chunks is below $\tau = 0.3$. Otherwise the query reaches the generator, which can still abstain if the context lacks the requested facts. Algorithm 1 details the pipeline.

4.6. Selection-matched text ablation design

To test whether gains come from workflow structuring or from procedure selection, *Selection-Matched Text RAG* (logged as *SOP-Oracle RAG*) uses the same selection rule (Eq. (1)) and the same query-expanded supporting retrieval as Workflow GraphRAG, but supplies the raw, unedited text chunk of p^* instead of the serialized workflow W. Both systems therefore receive the same procedure and supporting chunks and differ only in representation: structured workflow block versus original prose. Figure 6 shows the context supplied to each main comparison system.

Algorithm 1 Execution pipeline of Process-Aware Workflow GraphRAG

Require: Question q, workflow graph G, vector index I, threshold $\tau = 0.3$, retrieval depth $k = 3$

$p^* \leftarrow \arg\max_p s(q, p)$ {Evaluate selection via Eq. (1)}
if $s(q, p^*) < 3$ **then**
  $p^* \leftarrow$ none
end if
if $p^* \neq$ none **then**
  $W \leftarrow$ serialized ordered step sequence of p^* from G
  $q' \leftarrow q \,\|\, title_{p^*} \,\|\, actions_{1:3}(p^*)$ {Query expansion}
else
  $W \leftarrow \emptyset$
  $q' \leftarrow q$ {Retain unexpanded query}
end if
$C \leftarrow \textsc{RetrieveTopK}(I, q', k)$
$b \leftarrow \max_{c \in C} \textsc{CosineSimilarity}(q, c)$
if $p^* =$ none **and** $b < \tau$ **then**
  return Refusal response ("Information not available in official SOPs")
end if
return $\textsc{GenerateLocalLLM}(\textsc{PromptTemplate}(W \,\|\, C, q))$

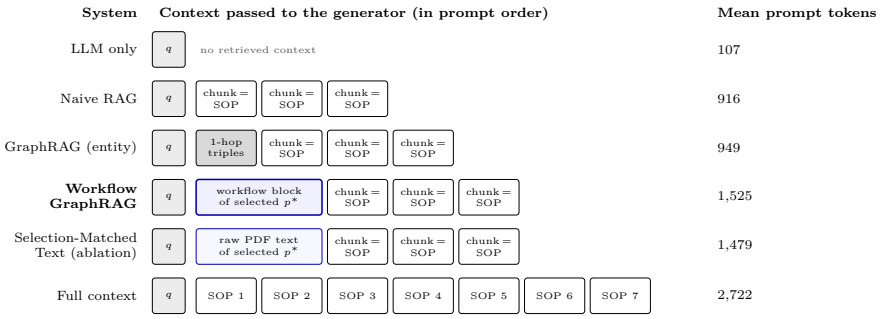


Figure 6: Context supplied to the generator by the main comparison systems. Each chunk is a complete SOP, so top-3 retrieval always adds three procedures. Workflow GraphRAG and the ablation share selection and supporting chunks and differ only in how p^* is represented. Token counts are means over all 40 questions, including the shared template.

5. Experimental setup

5.1. Evaluated systems

We evaluate 10 configurations in three groups (Table 4): four main systems (Naive RAG, the custom entity-graph GraphRAG, Workflow GraphRAG, and the selection-matched ablation); four supplementary RAG baselines (Hybrid RAG [5, 6], Agentic RAG [7], Corrective RAG [8], and Multimodal-Layout RAG); and two reference systems (LLM only, which shows parametric knowledge without retrieval, and full-context prompting, which supplies all 7 SOPs as an upper bound on available evidence).

Table 4: System configurations evaluated across 10 architectures. All systems share identical embedding models, chunking parameters (1,700/100 characters), vector stores, generator, and prompt templates.

System Configuration	Context Construction and Retrieval Mechanism
Main Comparison and Ablation	
Naive RAG	Top-3 dense chunks by cosine similarity; refuses if best similarity < 0.3.
GraphRAG (custom entity graph)	1-hop expansion over 17 curated relational triples; matched triples prepended to top-3 dense chunks.
Workflow GraphRAG (proposed)	Ordered workflow steps from directed graph with explicit actors and inputs, followed by top-3 supporting chunks.
Selection-Matched Text RAG (SOP-Oracle ablation)	Identical rule-based selection, providing selected raw SOP text in place of workflow block, followed by top-3 supporting chunks.
Supplementary RAG Baselines	
Hybrid RAG	Reciprocal rank fusion of top-5 dense and top-5 BM25 chunks ($k_{\text{rrf}} = 60$); top-3 retained.
Agentic RAG	ReAct loop executing up to 3 iterative search queries directed by generator.
Corrective RAG (CRAG)	Retrieval grading: ≥ 0.55 retained, 0.35–0.55 query rewritten, < 0.35 refused.
Multimodal-Layout RAG	Top-3 dense chunks augmented with structural document layout tags.
Reference Baselines	
LLM Only	Zero-shot generation without retrieved context.
Full-Context Prompting	All 7 SOP documents provided directly in generator prompt ($\approx$1,100 words).

5.2. Custom entity-graph baseline scope and validity threat

The entity-graph baseline is our own implementation with 17 curated relational triples (Supplementary Table S2), not a deployment of Microsoft GraphRAG [3], LightRAG [12], or HippoRAG [13]. It is a reference point for basic entity-triple augmentation rather than an evaluation of published GraphRAG frameworks, which we treat as a validity constraint in Section 7.

5.3. Shared infrastructure and experimental controls

All retrieval systems share the same infrastructure:

- **Embedding model:** `all-indo-e5-small-v4` [19], a 384-dimensional Indonesian sentence embedder fine-tuned on 50 procedural question-answer pairs with Multiple Negatives Ranking loss [21]; an ablation uses the non-fine-tuned base model.
- **Vector store:** Chroma, storing 7 chunks (1,700-character window, 100-character overlap).
- **Generator:** `gemma4:e2b` (5.1B parameters, Q4_K_M quantization) served by Ollama 0.35.1 at temperature 0 and seed 42, after one warm-up inference.
- **Hardware and logging:** an Apple M5 workstation (10 CPU cores, 24 GB unified memory) running macOS 26.6.2 on AC power; every generator call logs prompt tokens, completion tokens, and latency.

5.4. Evaluation benchmark

We built a held-out benchmark of 40 questions drafted from the source SOPs (Table 5). No item was generated by an evaluated system, and a lexical-overlap audit found no question with ROUGE-L ≥ 0.5 against the fine-tuning pairs. The benchmark has 35 answerable questions (32 single-SOP and 3 cross-SOP) and 5 unanswerable ones. Each answerable item defines a reference answer and required key facts; procedural items also specify the gold step sequence, actors, and matching keyword groups. The first author verified all items against the official PDFs. One example per category is given in Supplementary Table S3.

Table 5: Composition of the 40-question held-out institutional benchmark.

Category	n	Operational Target
Procedural	11	Complete step sequence, chronological order, and actors (7 formal, 4 colloquial).
Actor attribution	7	Identification of verifying, signing, or receiving officials.
Document requirements	7	Identification of mandatory forms, attachments, and outputs.
Time and location	7	Stated execution durations, service desks, and IT portals.
Cross-SOP discrimination	3	Differentiating easily confused procedures (evaluating E3).
Unanswerable / Refusal	5	Inquiries concerning out-of-scope administrative rules.
Total	40	

5.5. Evaluation metrics

All primary metrics are computed deterministically, without LLM judges:

- **Key-Fact Recall (KFR):** share of required reference facts detected in the response, over the 35 answerable questions.
- **ROUGE-1, ROUGE-2, ROUGE-L:** n-gram and longest-common-subsequence overlap with the reference answer [22].
- **Step Coverage:** on the 11 procedural questions, the share of gold steps S detected in the response ($|D|/|S|$); a step counts as detected when at least two of its keyword groups match.
- **Topological Inversion Rate (TIR):** share of detected step pairs (v_i, v_j) with $i < j$ that appear in reversed order:

$$\mathrm{TIR} = \frac{\left|\{(v_i, v_j) \in D^2 : i < j,\ pos(v_i) > pos(v_j)\}\right|}{\left|\{(v_i, v_j) \in D^2 : i < j,\ pos(v_i) \neq pos(v_j)\}\right|}, \tag{2}$$

  defined only when at least two comparable steps are detected.
- **Actor Mismatch Rate (AMR):** among detected steps with an explicitly named actor, the share attributed to the wrong official:

$$\mathrm{AMR} = \frac{mismatch}{correct + mismatch}. \tag{3}$$

- **Refusal accuracy and false refusal rate:** the share of the 5 unanswerable questions correctly refused, and the share of the 35 answerable questions incorrectly refused.
- **Resource costs:** prompt tokens, completion tokens, LLM invocations, and latency per question, averaged over all 40 items.

5.6. Manual audit of deterministic step scorer

Because step detection, TIR, and AMR rely on keyword matching, the first author checked 60 step-alignment rows, sampled across Naive RAG, GraphRAG, and Workflow GraphRAG (stratified, seed 42), against the full response texts (Table 6).

Table 6: Manual audit of deterministic step-alignment scorer across 60 sampled rows (Naive RAG, GraphRAG, and Workflow GraphRAG).

Measure	Agreement / κ	Labelled
Step detected	0.983 / 0.955	60/60
Actor status	0.933 / 0.835	60/60

Agreement was 0.983 ($\kappa = 0.955$) for step detection and 0.933 ($\kappa = 0.835$) for actor attribution. The single detection discrepancy (GraphRAG, late tuition payment UKT-P1, step 5) arose because the model put actor and action

on separate markdown lines ("**Pelaku:** Wakil Dekan Sumber Daya" followed by "**Tindakan:** Memproses…"), whereas the line-based scorer requires two keyword groups on one line. No genuine actor errors were found in the 60 rows (AMR = 0.000), which does not imply zero actor errors in uninspected outputs.

5.7. Secondary LLM-as-a-judge metrics

As supplementary corroboration, Ragas faithfulness and answer relevancy [23] were computed with an independent judge model (`llama3.1:8b`) [24]. Because of JSON parsing failures in the judge output, valid scores exist for 35/40 questions for Workflow GraphRAG and 26–27/40 for the baselines, so these metrics serve only as secondary evidence.

5.8. Statistical testing methodology

Metrics are reported as means with 95% bootstrap confidence intervals (10,000 resamples) [25]. Workflow GraphRAG is compared with baselines using two-sided paired Wilcoxon signed-rank tests [26], with Holm-Bonferroni correction [27] within each family of comparisons (against Naive RAG and against GraphRAG). Because unadjusted intervals and family-wise adjusted p-values rest on different multiplicity assumptions, an interval may exclude zero while the adjusted p-value exceeds $\alpha = 0.05$; we report both and do not describe non-significant results as equivalence.

6. Experimental results

All results come from the run on the 40 verified benchmark items (`results/full_v3_20261008/`, branch `exp/full-run-v1`).

6.1. Procedural answer quality and order preservation (RQ1)

Table 7 reports all 10 systems. Workflow GraphRAG, Selection-Matched Text RAG, and full-context prompting reach the highest key-fact recall: 0.955 [0.917, 0.990], 0.962 [0.914, 1.000], and 0.962 [0.914, 1.000]. Naive RAG reaches 0.712 [0.571, 0.840] and the entity-graph baseline 0.698 [0.560, 0.829]. Workflow GraphRAG attains 1.000 [1.000, 1.000] step coverage, compared with 0.727 for Naive RAG and 0.687 for GraphRAG.

Against GraphRAG, Workflow GraphRAG improves key-fact recall by +0.257 [0.133, 0.395] (Holm-adjusted $p = 0.034$) and ROUGE-L by +0.077 [0.031, 0.127] (Holm-adjusted $p = 0.038$); the step-coverage difference is not significant after adjustment ($p = 0.438$). Against Naive RAG, the recall gain of +0.243 [0.114, 0.381] has a Holm-adjusted $p = 0.053$ and therefore does not reach significance at $\alpha = 0.05$, although the unadjusted interval excludes zero; with $n = 40$, statistical power is limited.

On detected steps, Workflow GraphRAG, Naive RAG, Selection-Matched Text RAG, and the supplementary baselines show an observed TIR of 0.000, whereas the entity-graph baseline (0.083 [0.000, 0.250]), full-context prompting (0.027 [0.000, 0.082]), and LLM only (0.333 [0.000, 0.667]) show inversions.

Table 7: Summary of evaluation results across ten system configurations. KFR and false refusal: 35 answerable questions; Coverage, TIR, and AMR: 11 procedural questions; refusal accuracy: 5 unanswerable questions; prompt tokens: all 40. Point estimates; 95% confidence intervals are in Supplementary Table S1.

System	KFR ↑	Coverage ↑	TIR ↓	AMR ↓	Refusal acc. ↑	False ref. ↓	Prompt tok.
Naive RAG	0.712	0.727	0.000	0.000	0.600	0.000	916
GraphRAG (entity)	0.698	0.687	0.083	0.000	0.600	0.000	949
Workflow GraphRAG	0.955	1.000	0.000	0.000	1.000	0.000	1525
Selection-Matched Text RAG (ablation)	0.962	0.975	0.000	0.000	1.000	0.000	1479
Hybrid RAG	0.683	0.712	0.000	0.000	0.600	0.000	882
Agentic RAG	0.712	0.702	0.000	0.000	0.600	0.000	1815
Corrective RAG	0.569	0.636	0.000	0.000	1.000	0.371	725
Multimodal-layout RAG	0.702	0.702	0.000	0.000	0.600	0.000	994
LLM only	0.113	0.105	0.333	0.444	0.000	0.000	107
Full context	0.962	0.944	0.027	0.000	1.000	0.000	2722

AMR is 0.000 for every retrieval-augmented system and 0.444 for LLM only. Both metrics are conditional on detection, so systems differ mainly in completeness and recall rather than in step reordering.

By category (Table 8), Workflow GraphRAG reaches 1.000 key-fact recall on procedural questions (0.727 for both baselines) and 0.889 on cross-SOP questions (0.611). In cross-SOP questions about signatories in different procedures, its explicit step outputs allow unambiguous attribution, mitigating error E3.

Table 8: Key-fact recall by question category for the primary comparison systems on 40 verified questions. Procedural category also includes step coverage.

System	Procedural (KFR/Cov.)	Actor	Documents	Time/location	Cross-SOP
Naive RAG	0.727 / 0.727	0.857	0.619	0.679	0.611
GraphRAG (entity)	0.727 / 0.687	0.786	0.619	0.679	0.611
Workflow GraphRAG	1.000 / 1.000	0.952	0.905	0.964	0.889

6.2. Selection versus structuring ablation (RQ2)

Workflow GraphRAG (0.955 KFR) and Selection-Matched Text RAG (0.962 KFR) share the selection mechanism (Eq. (1)) and the top-3 supporting retrieval and differ only in whether the selected procedure is supplied as a workflow block or as raw PDF text. On the 35 answerable questions, a paired Wilcoxon signed-rank test gives a mean difference of -0.007 $[-0.036,\ 0.021]$ with $p = 0.59$, and scores differ on only 3 questions. The test does not detect a difference; $p = 0.59$ does not prove equivalence, but the data provide no evidence that graph serialization adds factual accuracy over the selected raw text on this benchmark. Figure 7 places this comparison alongside the comparisons against chunk retrieval.

The step-coverage gap (1.000 vs. 0.975) comes from a single item (UKT-P1): as in Section 5.6, the ablation put actors and actions on separate markdown lines, so the line-based scorer missed step co-occurrence although all 11 steps were present.

Table 9 reports the selector outcome on all 40 questions. All 32 answerable single-SOP questions were routed to the correct procedure, no question

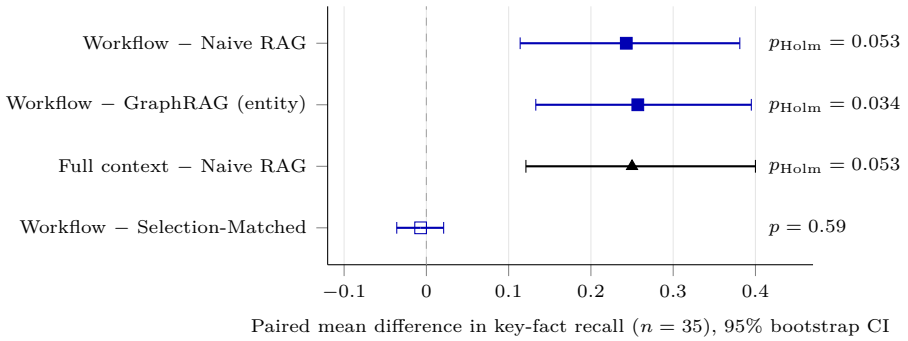


Figure 7: Paired differences in key-fact recall on the 35 answerable questions, with 95% bootstrap intervals and Wilcoxon p-values (Holm-adjusted within each reference family). With selection held fixed, the difference is near zero.

was misrouted between procedures, and for each cross-SOP question the selector chose one of the procedures involved. Ragas scores point in the same direction: `llama3.1:8b` rates Workflow GraphRAG at 0.805 faithfulness and 0.682 relevancy, compared with 0.762/0.549 for Naive RAG and 0.743/0.537 for GraphRAG.

Table 9: Outcome of the rule-based SOP selector (Eq. (1)) for Workflow GraphRAG on all 40 benchmark questions, read from the `matched_sop` field of the run log.

Question group	n	Selector outcome	Consequence
Answerable, single SOP	32	Correct SOP selected (32/32)	Workflow block of correct SOP in prompt
Answerable, cross-SOP (X-1–X-3)	3	One of the involved SOPs selected (X-1: Reactivation; X-2: Academic leave; X-3: Course registration)	Other SOPs reach the prompt only via supporting chunks
Unanswerable, SOP topic (NA-1–NA-3)	3	Topical SOP selected	Generator abstained (3/3)
Unanswerable, no match (NA-4, NA-5)	2	No SOP reaches $s \geq 3$	Deterministic refusal before generation
Total	40	No misrouting between procedures	

6.3. Refusal and boundary handling (RQ3)

Workflow GraphRAG, Selection-Matched Text RAG, and full-context prompting refuse all 5 unanswerable questions with no false refusals (Table 7). LLM only never abstains (0.000 refusal accuracy and 0.000 false refusals); the value of 0.333 in earlier drafts was a column transposition of its TIR. Corrective RAG refuses every unanswerable question but also 13 of 35 answerable ones (false refusal rate 0.371) because of its aggressive threshold ($\tau = 0.35$).

Naive RAG, GraphRAG, Hybrid, Agentic, and Multimodal-Layout RAG each score 0.600 refusal accuracy with no false refusals, partly as a scoring artefact. Below $\tau = 0.30$, Naive RAG returns a fixed template ("*Maaf, saya tidak menemukan jawaban pada dokumen SOP resmi yang tersedia*") without calling the generator, which happened on 10 queries (8 answerable, plus NA-4 and NA-5). The refusal patterns omitted "*tidak menemukan*", so the template was scored as a non-refusal: the 8 answerable queries received KFR = 0.000 without counting as false refusals, and NA-4 and NA-5 counted as failed refusals. Only NA-1 to NA-3, where the generator wrote "*tidak ditemukan*", were credited (3/5).

Workflow GraphRAG refused NA-4 and NA-5 before inference, since no SOP scored 3 and similarity was below 0.3. For NA-1 to NA-3 a topical SOP was selected and the generator stated that the requested fee or GPA criteria are not in the SOP. Unanswerable questions that mention SOP keywords thus reach the generator; this worked on the benchmark but is an operational boundary, not a general check of evidence sufficiency.

6.4. Computational efficiency and resource trade-offs (RQ4)

Table 10 summarises efficiency. Workflow GraphRAG uses 1,525 prompt tokens per query, more than Naive RAG (916) and GraphRAG (949) because it adds the workflow block, but 44% fewer than full-context prompting (2,722; about 56% of its volume) at numerically close recall (0.955 vs. 0.962). Figure 8 plots this trade-off for all systems.

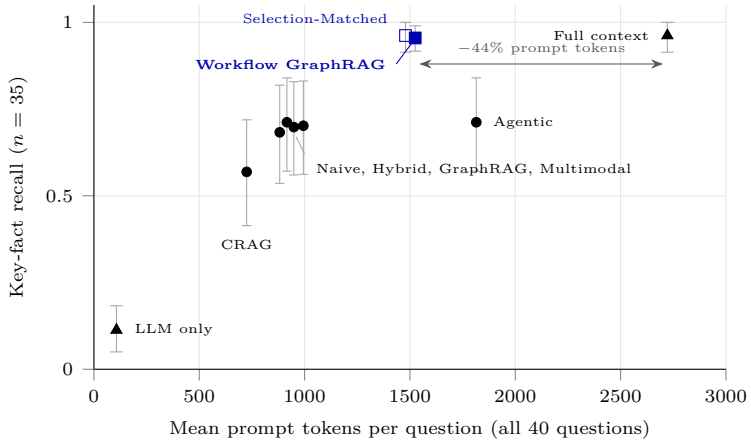


Figure 8: Key-fact recall against mean prompt tokens for all evaluated systems. Error bars: 95% bootstrap intervals. Circles: chunk-retrieval baselines; triangles: reference systems; squares: systems with rule-based SOP selection.

Mean generator calls reflect pre-generation refusal: chunk-based baselines skip the generator on 10 of 40 queries (30/40 = 0.75 calls), Workflow GraphRAG on 2 (38/40 = 0.95), and full-context prompting never (1.00); Agentic RAG averages 3.27 calls in its ReAct loop. Median latency is 10.7 s for Workflow GraphRAG, 9.9 s for Naive RAG, and 12.1 s for full-context prompting; Agentic

Table 10: Operational resource consumption per query across evaluated systems: mean prompt and completion tokens, mean generator invocations, and median execution latency across all 40 questions.

System	Prompt tok.	Completion tok.	LLM calls	Latency (s)
Naive RAG	916	555	0.75	9.9
GraphRAG (entity)	949	559	0.75	9.6
Workflow GraphRAG	1525	710	0.95	10.7
Hybrid RAG	882	566	0.75	9.3
Agentic RAG	1815	1857	3.27	28.7
Corrective RAG	725	661	1.05	12.7
Multimodal-layout RAG	994	561	0.75	9.8
LLM only	107	792	1.00	10.4
Full context	2722	792	1.00	12.1

RAG needs 1,815 prompt tokens, 1,857 completion tokens, and 28.7 s without improving recall (0.712 KFR). Local deployment removes per-token API charges but requires hardware, which we did not price.

6.5. Embedding sensitivity ablation

We repeated Naive RAG, GraphRAG, and Workflow GraphRAG with the non-fine-tuned base `all-indo-e5-small-v4` model instead of the fine-tuned checkpoint (Figure 9). With the base model, Naive RAG rises from 0.712 to 0.910 KFR (step coverage from 0.727 to 0.898) and GraphRAG from 0.698 to 0.900, while Workflow GraphRAG stays at 0.955. Overfitting to the phrasing of the 50 fine-tuning pairs is plausible, but objective mismatch between the contrastive loss and chunk retrieval, training instability on small samples, and distorted embedding geometry are untested alternatives. Either way, well-configured chunk retrieval is competitive on this corpus (0.910 KFR), which narrows the margin between conventional and workflow-guided retrieval.

6.6. Qualitative error analysis

Two examples illustrate behavioural differences:

1. **Cross-SOP disambiguation (X-2):** asked "Who signs the academic leave permit and late tuition permit letters, respectively?", Naive RAG retrieved both procedures but listed several candidate signatories under each heading (KFR = 0.5), whereas Workflow GraphRAG used step outputs to answer "Dean for Academic Leave Permit; Vice Dean for Resources for Late Tuition Permit" (KFR = 1.0).

2. **Omitted condition (IRS-A1):** for "Who approves the IRS form and how?", Workflow GraphRAG named the academic advisor and the SIAP portal but omitted that approval follows verification of the credit load (KFR = 0.667).

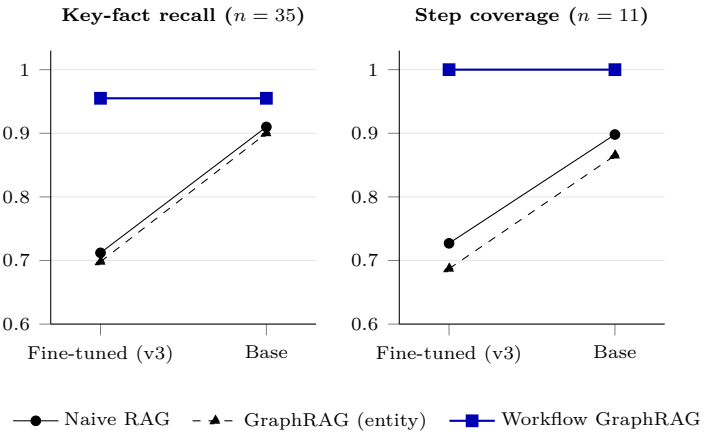


Figure 9: Embedding sensitivity: key-fact recall and step coverage with the fine-tuned (v3) and the non-fine-tuned base `all-indo-e5-small-v4` embedding. Chunk-retrieval baselines improve markedly with the base model, whereas Workflow GraphRAG is unchanged.

7. Discussion and threats to validity

7.1. Empirical findings and architectural interpretation

The results support two conclusions. First, the complete Workflow GraphRAG system, combining rule-based selection with structured workflow blocks, achieves high factual completeness (0.955 KFR) and step coverage (1.000), with recall close to full-context prompting at 44% fewer prompt tokens. Second, graph-derived serialization was *not* shown to add factual accuracy over the selected procedure's raw text ($p = 0.59$).

In concise, linear workflows, procedure-level selection is thus the key differentiator from unconstrained retrieval, which injects three competing procedures; routing rules, metadata filters, or workflow indexes can all provide it, although our design does not fully separate selection from the accompanying query expansion and context budget. The value of explicit structure lies in consistent formatting, compact prompts, and explicit deliverable attribution rather than in higher recall.

7.2. Threats to internal and comparative validity

- **Custom baseline:** the entity-graph baseline uses 17 hand-curated triples and does not represent automated frameworks such as Microsoft GraphRAG [3] or LightRAG [12]; the result $p = 0.034$ holds against this baseline, not against the GraphRAG paradigm.
- **Benchmark curation:** questions, reference answers, and gold steps were written and verified by the first author against the PDFs, not by an independent administrative panel.
- **Sample size:** with 40 questions, power is limited, as the borderline comparison with Naive RAG ($p = 0.053$) shows.

- **Audit scope:** the audit covered 60 sampled rows from three systems; it supports scorer reliability but cannot establish the absence of actor errors in uninspected outputs.

7.3. Corpus and generalizability limitations

- **Short linear procedures:** the corpus has 7 one-page procedures (1,097 words) without branching, parallel execution, or gateways; findings may not transfer to branching enterprise workflows, where BPMN gateways may be essential.
- **Selector transferability:** the keyword and alias rules are tailored to 7 SOPs; corpora of hundreds of procedures would need automated classification or hierarchical vector routing.
- **Supporting-chunk contamination:** the top-3 chunks appended after the workflow block reintroduce unselected SOPs; restricting them to the selected SOP should be tested.
- **Single generator:** only `gemma4:e2b` at temperature 0 was evaluated; other model families (e.g., Llama-3.2, Qwen-2.5) are needed to test model invariance.

8. Conclusion and future work

We compared SOP selection and workflow-graph structuring in local RAG across 10 configurations on 40 verified institutional questions with `gemma4:e2b`. For RQ1, Workflow GraphRAG reached 0.955 KFR and 1.000 step coverage, above naive (0.712; Holm $p = 0.053$) and entity-graph retrieval (0.698; Holm $p = 0.034$), with an observed TIR of 0.000. For RQ2, the selection-matched ablation reached 0.962 KFR with no detected difference ($p = 0.59$). For RQ3, all 5 unanswerable questions were refused. For RQ4, Workflow GraphRAG used 44% fewer prompt tokens than full-context prompting at close recall.

For practitioners deploying small models over concise procedures, procedure-level selection is the main lever against cross-document distractors. Workflow graphs add consistently ordered output and shorter prompts than full context, but no extra factual recall over well-selected raw text. Future work will replace heuristic selection with learned procedural classifiers, extend workflow representations to branching procedures parsed from BPMN XML, compare against published GraphRAG implementations (LightRAG, HippoRAG), and conduct an a priori powered evaluation with independent administrative validation.

Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The authors thank the Dean of the Faculty of Science and Mathematics, Universitas Diponegoro, Prof. Dr.

Kusworo Adi, the Head of the Department of Informatics, Dr. Aris Sugiharto, the Undergraduate Thesis Coordinator, Nurdin Bahtiar, and the thesis examiners, Dr. Sutikno and Yunila Dwi Putri Ariyanti, for their guidance and feedback, and the Faculty of Science and Mathematics for permitting this research to be carried out using its SOPs.

Data and code availability

Code, benchmark, and evaluation scripts are available at `https://github.com/reon01425-glitch/Chatbot-RAG-Comparison`, branch `exp/full-run-v1`. Experimental logs are deposited in `results/full_v3_20261008/` (main run) and `results/full_base_20261008/` (embedding ablation). The repository includes the 7 source SOP documents redistributed under institutional permission.

CRediT authorship contribution statement

Cipta Fikri Wiratama: Conceptualization, Methodology, Software, Data curation, Validation, Investigation, Writing – original draft. **Indra Waspada**: Conceptualization, Supervision, Writing – review & editing. **Dhena Kamalia Fu'adi**: Data curation, Validation, Writing – review & editing. **Ramona Ramli**: Conceptualization, Methodology, Supervision, Writing – review & editing. **Henri Tantyoko**: Methodology, Validation, Writing – review & editing. **Sandy Kurniawan**: Supervision, Methodology, Writing – review & editing. **Satriawan Rasyid Purnama**: Validation, Formal analysis, Writing – review & editing.

Ethics statement

This study did not involve human participants, personal data, or animals. The benchmark questions and reference answers were written by the authors, and the SOP documents were used with the permission of the Faculty of Science and Mathematics, Universitas Diponegoro.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI use

During the preparation of this work, the authors used generative AI tools to assist with productivity in literature search, drafting, and academic language editing. All methodological formulations, system implementations, and experimental analyses were designed by the authors, who reviewed and edited all generated text and assume full responsibility for the published content.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 9459–9474.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, arXiv:2312.10997 (2023).

[3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, J. Larson, From local to global: A graph RAG approach to query-focused summarization, arXiv:2404.16130 (2024).

[4] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, S. Tang, Graph retrieval-augmented generation: A survey, arXiv:2408.08921 (2024).

[5] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389.

[6] G. V. Cormack, C. L. A. Clarke, S. Büttcher, Reciprocal rank fusion outperforms Condorcet and individual rank learning methods, in: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2009, pp. 758–759.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations (ICLR), 2023.

[8] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, arXiv:2401.15884 (2024).

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations (ICLR), 2024.

[10] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Transactions of the Association for Computational Linguistics 12 (2024) 157–173.

[11] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (7) (2024) 3580–3599.

[12] Z. Guo, L. Xia, Y. Yu, T. Ao, C. Huang, LightRAG: Simple and fast retrieval-augmented generation, arXiv:2410.05779 (2024).

[13] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, Y. Su, HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024.

[14] L. Lin, Z. Zhu, T. Zhang, Y. Wen, SOPRAG: Multi-view graph experts retrieval for industrial standard operating procedures, arXiv:2602.01858 (2026).

[15] K. Wu, C. Kuai, Z. Li, J. Jiang, S. Shen, S. Wang, C.-W. Hu, Z. Tu, Y. Zhou, Knowledge is not static: Order-aware hypergraph RAG for language models, arXiv:2604.12185 (2026).

[16] M. Dumas, M. La Rosa, J. Mendling, H. A. Reijers, Fundamentals of Business Process Management, 2nd Edition, Springer, 2018.

[17] Object Management Group, Business Process Model and Notation (BPMN), Ver-

sion 2.0, OMG Standard formal/2011-01-03 (2011). Also published as ISO/IEC 19510:2013.

[18] F. Friedrich, J. Mendling, F. Puhlmann, Process model generation from natural language text, in: Advanced Information Systems Engineering (CAiSE 2011), Vol. 6741 of Lecture Notes in Computer Science, Springer, 2011, pp. 482–496.

[19] LazarusNLP, all-indo-e5-small-v4, Hugging Face model card, `https://huggingface.co/LazarusNLP/all-indo-e5-small-v4`, accessed May 2026.

[20] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, F. Wei, Text embeddings by weakly-supervised contrastive pre-training, arXiv:2212.03533 (2022).

[21] M. Henderson, R. Al-Rfou, B. Strope, Y.-H. Sung, L. Lukács, R. Guo, S. Kumar, B. Miklos, R. Kurzweil, Efficient natural language response suggestion for smart reply, arXiv:1705.00652 (2017).

[22] C.-Y. Lin, ROUGE: A package for automatic evaluation of summaries, in: Text Summarization Branches Out, Association for Computational Linguistics, 2004, pp. 74–81.

[23] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, RAGAs: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 2024, pp. 150–158.

[24] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track, 2023.

[25] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall/CRC, 1993.

[26] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[27] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.